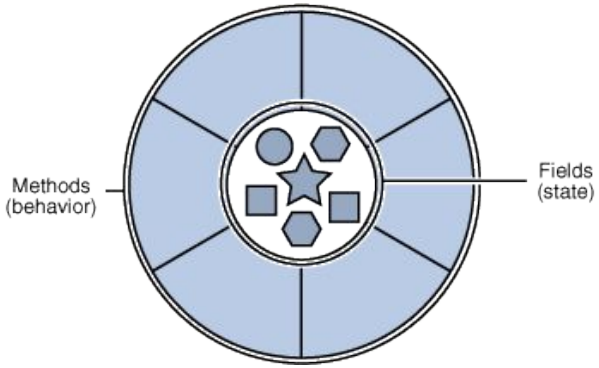
Classes, Objects and Constructor

Objects

- **object:** An entity that encapsulates data and behavior.
  - *data*: variables inside the object
  - *behavior*: methods inside the object
    - You interact with the methods; the data is hidden in the object.



- Constructing (creating) an object:

```
Type objectName = new Type(parameters);
```

- Calling an object's method:

```
objectName.methodName(parameters);
```

Classes

- **class**: A program entity that represents either:
  1. A program / module, or
  2. A template for a new type of objects.

– **object-oriented programming (OOP)**: Programs that perform their behavior as interactions between objects.

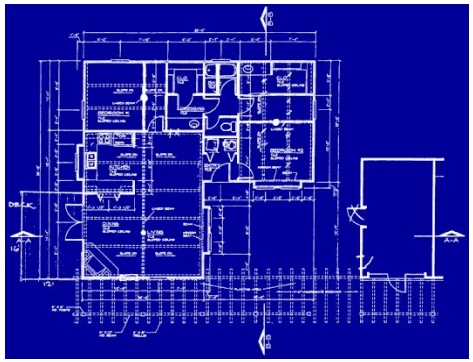
  – **abstraction**: Separation between concepts and details.
  Objects and classes provide abstraction in programming.

Blueprint analogy

iPod blueprint

state:
- current song
- volume
- battery life

behavior:
- power on/off
- change station/song
- change volume
- choose random song



creates

iPod #1

state:
- song = "1,000,000 Miles"
- volume = 17
- battery life = 2.5 hrs

behavior:
- power on/off
- change station/song
- change volume
- choose random song



iPod #2

state:
- song = "Letting You"
- volume = 9
- battery life = 3.41 hrs

behavior:
- power on/off
- change station/song
- change volume
- choose random song



iPod #3

state:
- song = "Discipline"
- volume = 24
- battery life = 1.8 hrs

behavior:
- power on/off
- change station/song
- change volume
- choose random song



`Point` objects

```
import java.awt.*;
...
Point p1 = new Point(5, -2);
Point p2 = new Point();          // origin (0, 0)
```

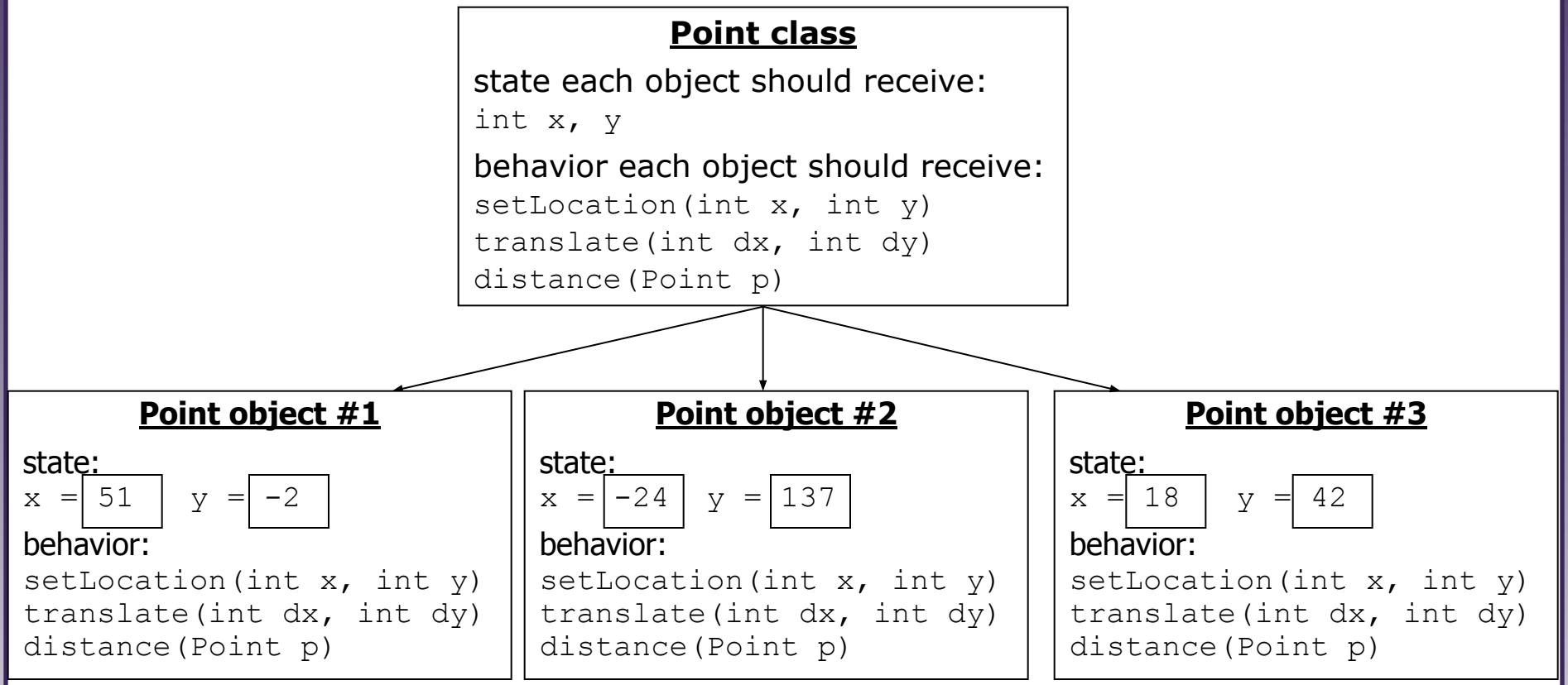
- Data:

Name	Description
`x`	the point's x-coordinate
`y`	the point's y-coordinate

- Methods:

Name	Description
`setLocation(`**`x, y`**`)`	sets the point's x and y to the given values
`translate(`**`dx, dy`**`)`	adjusts the point's x and y by the given amounts
`distance(`**`p`**`)`	how far away the point is from point *p*

`Point` class as blueprint



- The class (blueprint) describes how to create objects.
- Each object contains its own data and methods.
  - The methods operate on that object's data.

Fields

- **field**: A variable inside an object that is part of its state.
  - Each object has *its own copy* of each field.

- Declaration syntax:

```
private type name;
```

  - Example:

```
public class Point {
    private int x;
    private int y;

    ...
}
```

Instance methods

- **instance method** (or **object method**): Exists inside each object of a class and gives behavior to each object.

```
public type name(parameters) {
    statements;
}
```

  - same syntax as static methods, but without `static` keyword

  Example:

```
public void tranlate(int dx, int dy) {
    x += dx;
    y += dy;
}
```

Categories of methods

- **accessor**: A method that lets clients examine object state.
  - Examples: `distance`, `distanceFromOrigin`
  - often has a non-`void` return type

- **mutator**: A method that modifies an object's state.
  - Examples: `setLocation`, `translate`

- **helper**: Assists some other method in performing its task.
  - often declared as private so outside clients cannot call it

Constructors

- **Constructor** is a block of code that initializes the newly created object.
- A **constructor** resembles(is like) an instance method in **java** but it's not a method as it doesn't have a return type.
- **Constructor** has same name as the class and looks like this in a **java** code.
- A **constructor** is a special method that is used to initialize a newly created object and is called just after the memory is allocated for the object.
- It can be used to initialize the objects to desired values or default values at the time of object creation

Rules for writing Constructor:

1. Constructor(s) of a class must has same name as the class name in which it resides.
2. A constructor in Java can not be abstract, final, static and Synchronized.
3. Access modifiers can be used in constructor declaration to control its access i.e which other class can call the constructor.

Constructors

- **constructor**: Initializes the state of new objects.

```
public type(parameters) {
    statements;
}
```

  - runs when the client uses the new keyword
  - no return type is specified; implicitly "returns" the new object

```
public class Point {
    private int x;
    private int y;

    public Point(int initialX, int initialY) {
        x = initialX;
        y = initialY;
    }
```

Multiple constructors

- A class can have multiple constructors.
  - Each one must accept a unique set of parameters.

- *Example:* A `Point` constructor with no parameters that initializes the point to (0, 0).

```
// Constructs a new point at (0, 0).
public Point() {
    x = 0;
    y = 0;
}
```

Why constructor has no return type?

- When you create a object so **constructor** will automatically call according to which object is call.
- So the reason the **constructor** doesn't **return** a **value is** because it's **not** called directly by your code, **it's called by the memory allocation and object initialization code in the runtime.**
- Its **return value** (if it actually has one when compiled down to machine code) is opaque(Fuzzy) to the user - therefore, you can't specify it.
- The purpose of **constructor** is to initialize the object of a class while the purpose of a method is to perform a task by executing java code

Is constructor private?

- **Constructors**, like regular methods, can also be declared as **private**.
- You may wonder why we need a **private constructor** since it is only accessible from its own class.
- When a class needs to prevent the caller from creating objects. **Private constructors** are suitable.

The keyword `this`

- **this** : Refers to the implicit parameter inside your class.

  (a variable that stores the object on which a method is called)

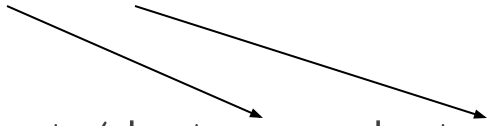
  - Refer to a field: this.**field**
  - Call a method: this.**method**(**parameters**);
  - One constructor can call another: this(**parameters**);

Calling another constructor

```
public class Point {
    private int x;
    private int y;

    public Point() {
        this(0, 0);
    }

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
    ...
}
```



- Avoids redundancy between constructors
- Only a constructor (not a method) can call another constructor

Calling another constructor

- In Java **another constructor** of the same class can be called from a **constructor** via this() .
- Note however that this has to be on the first line.
- Within a **constructor**, you can use the this keyword to **invoke another constructor** in the same class.
- Doing so is called an explicit **constructor** invocation.

Summary(Constructors)

- Constructors
  - Initialize an object of a class
  - Java requires a constructor for every class
  - Java will provide a default no-argument constructor if none is provided
  - Called when keyword new is followed by the class name and parentheses

19